3. **A commercial bank receives thousands of loan applications every month and wants to reduce the risk of defaults by automating its approval process. Using historical applicant data containing demographic and financial attributes, identify and implement an appropriate classification algorithm that predicts whether an applicant will default on a loan, and evaluate the model's reliability using standard performance metrics.**

   Logistic Regression is a supervised machine learning classification technique used to predict a categorical outcome. It is suitable for binary classification problems where the output has two classes. In this experiment, it predicts whether a loan applicant will default or not default based on demographic and financial attributes.

- Creates a realistic loan applicant dataset with demographic and financial attributes.
- Uses Logistic Regression to classify applicants as Default or No Default.
- Splits the data into training and testing sets and standardizes the features.
- Evaluates the model using Accuracy, Precision, Recall, and F1-score.
- Uses a Confusion Matrix to analyze correct and incorrect predictions.

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

np.random.seed(42)
n = 500

# Create meaningful loan applicant data
df = pd.DataFrame({
    'Age': np.random.randint(21, 65, n),
    'Income': np.random.randint(20000, 100000, n),
    'Loan_Amount': np.random.randint(5000, 50000, n),
    'Credit_Score': np.random.randint(300, 850, n),
    'Existing_Loans': np.random.randint(0, 5, n),
    'Employment_Years': np.random.randint(0, 30, n),
    'Debt_to_Income': np.round(np.random.uniform(0.1, 0.8, n), 2),
    'Savings': np.random.randint(5000, 100000, n)
```

```
})

# Generate probability of default
risk = (
    -0.004 * df['Credit_Score']
    + 0.8 * df['Debt_to_Income']
    + 0.15 * df['Existing_Loans']
    - 0.00001 * df['Savings']
    + 0.00001 * df['Loan_Amount']
)

probability = 1 / (1 + np.exp(-risk))
df['Default'] = np.random.binomial(1, probability)

print("Loan Applicant Dataset:")
print(df.head(10).to_string(index=False))

# Features and target
X = df.drop('Default', axis=1)
y = df['Default']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Performance metrics
print("\nPerformance Metrics:")
print("Accuracy :", round(accuracy_score(y_test, y_pred), 3))
print("Precision:", round(precision_score(y_test, y_pred), 3))
```

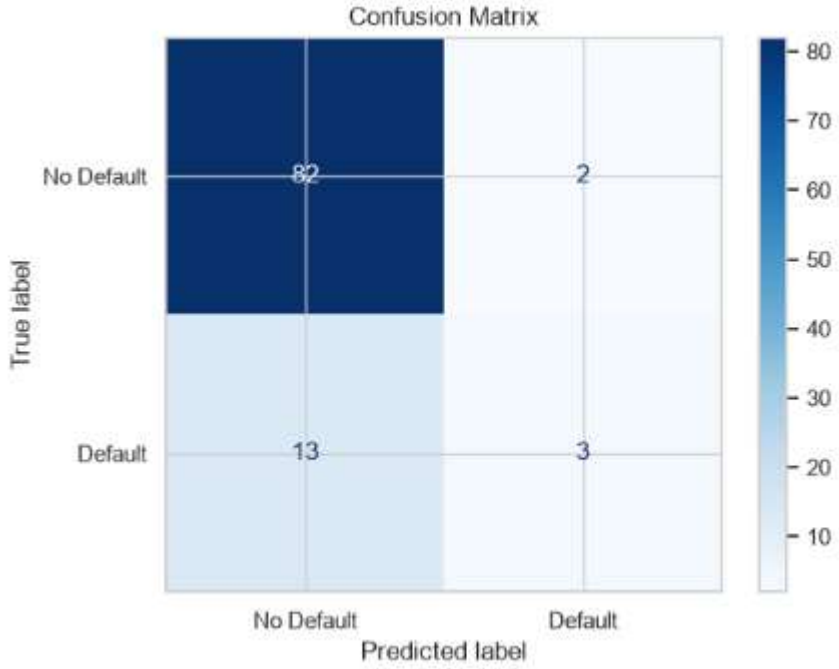
```
print("Recall   :", round(recall_score(y_test, y_pred), 3))
print("F1 Score :", round(f1_score(y_test, y_pred), 3))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['No Default', 'Default']
)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

Output

```
Loan Applicant Dataset:
 Age  Income  Loan_Amount  Credit_Score  Existing_Loans  Employment_Years  Debt_to_Income  Savings  Default
  59   40358        25609           469               4                24            0.69    35873        0
  49   23267        21478           655               3                28            0.30    63627        0
  35   58513        40442           490               2                 8            0.42    71155        0
  63   46092        44666           721               4                 0            0.41    30372        0
  28   31338        15147           306               3                17            0.25     8642        1
  41   20412        38561           432               3                20            0.38    33001        0
  59   27543        39884           330               0                 3            0.13    39064        0
  39   53496        10779           834               0                 9            0.29    99826        0
  43   80692        17298           725               4                24            0.67    19122        1
  31   64425        19153           693               3                 6            0.30    29820        0

Performance Metrics:
Accuracy : 0.85
Precision: 0.6
Recall   : 0.188
F1 Score : 0.286
```


Confusion Matrix
No Default
Default
True label
82
2
13
3
No Default
Default
Predicted label
80
70
60
50
40
30
20
10